

compatibility with all the tools associated with LaTeX, which makes some knowledge of PGF/TikZ very essential.

A simple PGF/TikZ example

PGF/TikZ produces vector graphics from a geometric description of the image to be drawn by using its own set of instructions. Before exploring the code any further, let us discuss two important aspects of PGF/TikZ, which are points and paths. Points refer to a specific location in a two-dimensional plane. Points can be specified as Cartesian coordinates, polar coordinates, named points and relative points in PGF/TikZ. To make our discussion relatively simple and less mathematical, we will only discuss the points being described as Cartesian coordinates. For example, the Cartesian coordinates (p, q) refer to a point located at p units in the direction of the x-axis and q units in the direction of the y-axis. For ease of understanding, the origin denoted by $(0,0)$ can be considered as located at the bottom-left corner of the plane when displayed on the screen.

The other important aspect of PGF/TikZ is how a path is traced in a two-dimensional plane. This aspect of tracing paths in PGF/TikZ is explained with the following example scripts. Let us start with a simple example involving PGF/TikZ code to understand how paths are specified. Consider the LaTeX script *pgf1.tex* shown below, which draws two rectangles. As stated earlier, the PGF/TikZ code is incorporated inside the minimal LaTeX template mentioned earlier.

```
\documentclass{article}
\usepackage{tikz}
\begin{document}
\begin{tikzpicture}
    \draw (1,1)--(2,1)--(2,3)--(1,3)--cycle;
    \draw (4,4)rectangle(5,6);
\end{tikzpicture}
\end{document}
```

Now let us go through the code in the file *pgf1.tex*, line-by-line, to understand it better. I will only explain the lines of code newly introduced in this script. The line of code `\usepackage{tikz}` makes sure that LaTeX can use all the functionalities of the package TikZ. The lines of code `\begin{tikzpicture}` and `\end{tikzpicture}` denote the environment inside which PGF/TikZ code can be entered to draw images. The line of code `\draw (1,1)--(2,1)--(2,3)--(1,3)--cycle;` draws four lines in the two-dimensional plane by using the *draw* command. The first line is drawn from point $(1,1)$ to point $(2,1)$, the second line is drawn from point $(2,1)$ to point $(2,3)$, the third line is drawn from point $(2,3)$ to point $(1,3)$, and the fourth line is drawn from point $(1,3)$ to point $(1,1)$. The portion of the code that is responsible for drawing the fourth line is `(1,3)--cycle;` which will complete the rectangle by drawing a line from point $(1,3)$ to the first point used in this *draw*

command, which is point $(1,1)$.

The line of code `\draw (4,4)rectangle(5,6);` draws a rectangle by providing the location of the bottom-left and

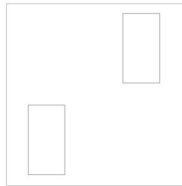


Figure 1: Two rectangles drawn with PGF/TikZ

top-right corners of the rectangle. In this case, the points are $(4,4)$ and $(5,6)$, respectively which represent these two corners. Please note the fact that PGF/TikZ commands are terminated with a semicolon (;), failing which they will lead to an error. Now let us execute this script

to view the image drawn. This LaTeX script can also be executed like any other ordinary LaTeX document — either use the command *pdflatex* or *latex* on a terminal or use Texmaker. I have used the command *pdflatex pgf1.tex* on the terminal to produce the output file *pgf1.pdf*. Figure 1 shows the image in the document *pgf1.pdf*.

Coloured images with PGF/TikZ

It is very easy to draw coloured images with PGF/TikZ.

Consider the script *pgf2.tex* given below. In this script, three triangles are drawn in red, green and blue.

```
\documentclass{article}
\usepackage{tikz}
\begin{document}
\begin{tikzpicture}
    \draw[fill=red] (0,1)--(2,1)--(1,3)--cycle;
    \draw[fill=green] (3,1)--(5,1)--(4,3)--cycle;
    \draw[fill=blue] (6,1)--(8,1)--(7,3)--cycle;
\end{tikzpicture}
\end{document}
```

The only line of code that needs an explanation is `\draw[fill=red] (0,1)--(2,1)--(1,3)--cycle;`. This line draws a triangle by drawing three lines. The first line is drawn from point $(0,1)$ to point $(2,1)$, the second line is drawn from point $(2,1)$ to point $(1,3)$, and the third line is drawn from point $(1,3)$ to point $(0,1)$, thus forming a triangle. The colour to be filled inside the triangle is provided as an option to the *draw* command. The option to fill the triangle with red colour is given by the code `[fill=red]`. The remaining two lines in the *tikzpicture* environment of LaTeX draw two more triangles, and fill them with green and blue. On executing the command *pdflatex pgf2.tex*, an output file called *pgf2.pdf* will be generated. Figure 2 shows the image in the document *pgf2.pdf*.

There are other options also available with the *draw*